RAMAIAH Institute of Technology

Department of Computer Science & Engineering

Course Code:	**CSL37**	**TERM: Oct 2024 – Jan 2025**	
Course Name:	**Object Oriented Programing Laboratory**	**Faculty In-charge:**	**Jamuna S Murthy**
Credits:	**0:0:1**	**SEMESTER :**	**III**

Design, develop, and implement the following programs in Java

	PART - A	CO	PO/PSO

1.	Design and implement a flight booking system using inheritance, showcasing **different types of constructors.** The system should include a base class **Flight** and two derived classes: **DomesticFlight** and **InternationalFlight.**
	1. Base Class **Flight**:
	○ Attributes: **String flightNumber, String airline, double basePrice.**
	○ Constructors: Use Default and parameterized.
	○ Method: **void displayDetails()** to print flight details.
	2. Derived Class **DomesticFlight:**
	○ Attribute: **double domesticTaxRate.**
	○ Constructors: Use **super** to call base class constructors and add a custom constructor.
	○ Methods:
	■ Override **displayDetails()** to include the total price.
	■ **double calculatePrice()** computes total price.
	3. Derived Class **InternationalFlight:**
	○ Attributes: **double internationalTaxRate, double customsFee.**
	○ Constructors: Use **super** and add a custom constructor.
	○ Methods:
	■ Override **displayDetails()** to include the total price.
	■ **double calculatePrice()** computes total price.
	4. Main Method:
	○ Create objects using different constructors.
	○ Use dynamic dispatch to call **displayDetails()** for both classes.
	// Base class Flight
	class Flight {
	protected String flightNumber;
	protected String airline;
	protected double basePrice;
	
	// Default Constructor
	public Flight() {
	this.flightNumber = "Unknown";
	this.airline = "Unknown";
	this.basePrice = 0.0;
	}
		1	2,3,5/3

```
    // Parameterized Constructor
    public Flight(String flightNumber, String airline, double basePrice) {
        this.flightNumber = flightNumber;
        this.airline = airline;
        this.basePrice = basePrice;
    }
    // Method to display flight details
    public void displayDetails() {
        System.out.println("Flight Number: " + flightNumber);
        System.out.println("Airline: " + airline);
        System.out.println("Base Price: $" + basePrice);
    }
}
// Derived class DomesticFlight
class DomesticFlight extends Flight {
    private double domesticTaxRate;
    // Constructor using super
     public DomesticFlight(String flightNumber, String airline, double basePrice, double
domesticTaxRate) {
        super(flightNumber, airline, basePrice);
        this.domesticTaxRate = domesticTaxRate;
    }
    // Method to calculate total price
    public double calculatePrice() {
        return basePrice + (basePrice * domesticTaxRate / 100);
    }
    // Overriding displayDetails method
    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Domestic Tax Rate: " + domesticTaxRate + "%");
        System.out.println("Total Price: $" + calculatePrice());
    }
}

// Derived class InternationalFlight
class InternationalFlight extends Flight {
    private double internationalTaxRate;
    private double customsFee;
    // Constructor using super
        public InternationalFlight(String flightNumber, String airline, double basePrice,
double internationalTaxRate, double customsFee) {
        super(flightNumber, airline, basePrice);
        this.internationalTaxRate = internationalTaxRate;
        this.customsFee = customsFee;
    }
    // Method to calculate total price
    public double calculatePrice() {
        return basePrice + (basePrice * internationalTaxRate / 100) + customsFee;
    }
    // Overriding displayDetails method
    @Override
    public void displayDetails() {
```

```
        super.displayDetails();
        System.out.println("International Tax Rate: " + internationalTaxRate + "%");
        System.out.println("Customs Fee: $" + customsFee);
        System.out.println("Total Price: $" + calculatePrice());
    }
}
// Main class
public class FlightBookingSystem {
    public static void main(String[] args) {
        // Creating Domestic Flight object
        Flight domesticFlight = new DomesticFlight("D123", "Air India", 500, 10);
        System.out.println("\nDomestic Flight Details:");
        domesticFlight.displayDetails();

        // Creating International Flight object
            Flight internationalFlight = new InternationalFlight("I456", "British Airways",
1000, 15, 50);
        System.out.println("\nInternational Flight Details:");
        internationalFlight.displayDetails();
    }
}
```

2.	Design a Java application for a car rental service using interfaces, focusing on **objects as parameters**, **argument passing**, and **returning objects**. The application should allow users to:
	1. **Define an Interface RentalService:**
	○ Methods: **checkAvailability(), rent(), returnCar(), and calculateRentalCost(int days, double distance).**
	2. **Create a Class Car Implementing RentalService:**
	○ Attributes: **make, model, year, dailyRate, isAvailable.**
	○ Methods:
	■ Constructor to initialize attributes.
	■ Implement interface methods.
	■ Add **displayDetails()** to show car details.
	■ Add **compareRates(Car other)** to return the car with the lower rate.
	3. **Create a Class CarRentalService:**
	○ Attributes: A list of **Car** objects.
	○ Methods:
	■ **addCar(Car car):** Adds a car to the service.
	■ **findCar(String make, String model):** Returns a car based on make and model.
	■ **displayAvailableCars():** Shows all available cars.
	■ Menu-based options: Check availability, rent a car, return a car, compare rates, and calculate rental cost.
	4. **Main Method:** Demonstrate the use of **objects as parameters**, **argument passing**, and **returning objects** through a menu-based system for the car rental service.
	`import java.util.ArrayList;`
	
	`// Define the RentalService interface`
	`interface RentalService {`
	`boolean checkAvailability();`
		1	2,3,5/3

```
    void rent();
    void returnCar();
    double calculateRentalCost(int days, double distance);
}

// Implement the Car class
class Car implements RentalService {
    private String make, model;
    private int year;
    private double dailyRate;
    private boolean isAvailable;

    public Car(String make, String model, int year, double dailyRate) {
        this.make = make;
        this.model = model;
        this.year = year;
        this.dailyRate = dailyRate;
        this.isAvailable = true;
    }

    @Override
    public boolean checkAvailability() {
        return isAvailable;
    }

    @Override
    public void rent() {
        if (isAvailable) {
            isAvailable = false;
        }
    }

    @Override
    public void returnCar() {
        isAvailable = true;
    }

    @Override
    public double calculateRentalCost(int days, double distance) {
        return (days * dailyRate) + (0.2 * distance);
    }

    public void displayDetails() {
         System.out.println(year + " " + make + " " + model + " - Daily Rate: $" + dailyRate
+ " - Available: " + isAvailable);
    }

    public Car compareRates(Car other) {
        return this.dailyRate < other.dailyRate ? this : other;
    }

    public String getMake() { return make; }
    public String getModel() { return model; }
```

```
}

// CarRentalService class
class CarRentalService {
    private ArrayList<Car> cars;

    public CarRentalService() {
        this.cars = new ArrayList<>();
    }

    public void addCar(Car car) {
        cars.add(car);
    }

    public Car findCar(String make, String model) {
        for (Car car : cars) {
                                                if    (car.getMake().equalsIgnoreCase(make)    &&
car.getModel().equalsIgnoreCase(model)) {
                return car;
            }
        }
        return null;
    }

    public void displayAvailableCars() {
        for (Car car : cars) {
            if (car.checkAvailability()) {
                car.displayDetails();
            }
        }
    }
}

// Main class with predefined operations
public class CarRentalApplication {
    public static void main(String[] args) {
        CarRentalService rentalService = new CarRentalService();

        // Adding sample cars
        Car car1 = new Car("Toyota", "Camry", 2022, 50);
        Car car2 = new Car("Honda", "Civic", 2021, 45);
        Car car3 = new Car("Ford", "Focus", 2023, 55);

        rentalService.addCar(car1);
        rentalService.addCar(car2);
        rentalService.addCar(car3);

        System.out.println("Available cars before renting:");
        rentalService.displayAvailableCars();

        // Rent a car
        car1.rent();
```

	System.out.println("\nAvailable cars after renting Toyota Camry:");
	rentalService.displayAvailableCars();
	
	// Return a car
	car1.returnCar();
	
	System.out.println("\nAvailable cars after returning Toyota Camry:");
	rentalService.displayAvailableCars();
	
	// Compare rental rates
	Car cheaperCar = car1.compareRates(car2);
	System.out.println("\nCheaper car between Toyota Camry and Honda Civic:");
	cheaperCar.displayDetails();
	
	// Calculate rental cost
	double cost = car2.calculateRentalCost(5, 100);
	System.out.println("\nRental cost for Honda Civic for 5 days and 100 miles: $" + cost);
	}
	}		

3.	Create a program to simulate a weather monitoring system using inheritance, dynamic method dispatch, and **String and StringBuffer handling**. The program should have a base class **WeatherStation** and two derived classes: **TemperatureStation** and **RainfallStation.** 1. **Base Class: WeatherStation** ○ **Attributes: String location, String stationId** ○ **Method: void displayData()**: Print generic weather station data. Add methods demonstrating String handling, such as concatenation, substring extraction, and comparison. 2. **Derived Class: TemperatureStation** ○ **Attribute: double temperature** ○ **Methods:** Override **displayData()** to include temperature details. Add a method demonstrating **StringBuffer** usage (e.g., appending, replacing, and reversing a string). 3. **Derived Class: RainfallStation** ○ **Attribute: double rainfall** ○ **Methods:** Override **displayData()** to include rainfall data. Add a method demonstrating another aspect of **StringBuffer** handling (e.g., inserting, deleting, or replacing). 4. **Main Method:** ○ Use an array of **WeatherStation** references to store objects of both **TemperatureStation** and **RainfallStation.** ○ Call **displayData()** dynamically for each object. ○ Demonstrate String and StringBuffer handling explicitly by: Creating a S**tring** to represent the weather station's name and manipulating it using **String** methods (e.g., concatenation, case conversion, substring). Using a	1	2,3,5/3

S**tringBuffer** to modify station details dynamically, such as appending data or reversing station IDs.

```
// Base class: WeatherStation
class WeatherStation {
    protected String location;
    protected String stationId;

    public WeatherStation(String location, String stationId) {
        this.location = location;
        this.stationId = stationId;
    }

    public void displayData() {
        System.out.println("Weather Station Location: " + location);
        System.out.println("Station ID: " + stationId);
    }

    public void stringOperations() {
        // Demonstrating String handling
        String stationInfo = "Station: " + stationId + " at " + location;
        System.out.println("Concatenation: " + stationInfo);
                System.out.println("Substring (first 10 chars): " + stationInfo.substring(0,
Math.min(10, stationInfo.length())));
                            System.out.println("Comparison (equals 'TestStation'): " +
stationId.equals("TestStation"));
    }
}

// Derived Class: TemperatureStation
class TemperatureStation extends WeatherStation {
    private double temperature;

    public TemperatureStation(String location, String stationId, double temperature) {
        super(location, stationId);
        this.temperature = temperature;
    }

    @Override
    public void displayData() {
        super.displayData();
        System.out.println("Temperature: " + temperature + "°C");
    }

    public void stringBufferOperations() {
        // Demonstrating StringBuffer usage
        StringBuffer sb = new StringBuffer(stationId);
        sb.append("_Temp");
        System.out.println("StringBuffer Append: " + sb);
        sb.replace(0, 3, "TMP");
        System.out.println("StringBuffer Replace: " + sb);
        sb.reverse();
        System.out.println("StringBuffer Reverse: " + sb);
```

```
    }
}

// Derived Class: RainfallStation
class RainfallStation extends WeatherStation {
    private double rainfall;

    public RainfallStation(String location, String stationId, double rainfall) {
        super(location, stationId);
        this.rainfall = rainfall;
    }

    @Override
    public void displayData() {
        super.displayData();
        System.out.println("Rainfall: " + rainfall + " mm");
    }

    public void stringBufferOperations() {
        // Demonstrating another StringBuffer operation
        StringBuffer sb = new StringBuffer(stationId);
        sb.insert(3, "_Rain");
        System.out.println("StringBuffer Insert: " + sb);
        sb.delete(3, 8);
        System.out.println("StringBuffer Delete: " + sb);
    }
}

// Main class
public class WeatherMonitoringSystem {
    public static void main(String[] args) {
        WeatherStation[] stations = new WeatherStation[2];
        stations[0] = new TemperatureStation("New York", "NY123", 25.5);
        stations[1] = new RainfallStation("Los Angeles", "LA789", 120.4);

        for (WeatherStation station : stations) {
            station.displayData();
            station.stringOperations();

            // Checking for StringBuffer operations dynamically
            if (station instanceof TemperatureStation) {
                ((TemperatureStation) station).stringBufferOperations();
            } else if (station instanceof RainfallStation) {
                ((RainfallStation) station).stringBufferOperations();
            }
            System.out.println("---------------------------");
        }
    }
}
```

4.	Implement a simple restaurant ordering system using inheritance, demonstrating the usage of **final and super.** The program should consist of a base class **MenuItem** and two derived classes: **FoodItem and DrinkItem.**
	
	1. **Create a base class MenuItem** with the following:
	○ **Attributes: final String name** and **double price.**
	○ **Constructor:** Initialize **name** and **price** attributes.
	○ **Method: void displayDetails()**: Prints basic menu item details. Mark the **displayDetails()** method as final to prevent overriding.
	2. **Create a derived class FoodItem** with the following:
	○ **Attribute: boolean isVegetarian.**
	○ **Constructor:** Initialize **name, price,** and **isVegetarian** attributes.
	○ **Method:** Add a **displayFoodDetails()** method that calls the **super.displayDetails()** method and includes whether the food item is vegetarian.
	3. **Create another derived class DrinkItem** with the following:
	○ **Attribute: boolean isAlcoholic.**
	○ **Constructor:** Initialize **name, price, and isAlcoholic** attributes.
	○ **Method:** Add a **displayDrinkDetails()** method that calls the **super.displayDetails()** method and includes whether the drink is alcoholic.
	4. In the **main** method Create an array of **MenuItem** references and store objects of both **FoodItem and DrinkItem.** Use **dynamic method dispatch** to call the **displayDetails()** method for each item. Demonstrate the use of **final** and **super** in the overridden methods.
	
	`// Base class representing a menu item`
	`class MenuItem {`
	`  final String name;  // Final attribute`
	`  double price;`
	
	`  // Constructor`
	`  public MenuItem(String name, double price) {`
	`  this.name = name;`
	`  this.price = price;`
	`  }`
	
	`  // Final method to display menu item details`
	`  public final void displayDetails() {`
	`  System.out.println("Item: " + name + ", Price: $" + price);`
	`  }`
	`}`
	
	`// Derived class representing a food item`
	`class FoodItem extends MenuItem {`
	`  boolean isVegetarian;`
	
	`  // Constructor`
	`  public FoodItem(String name, double price, boolean isVegetarian) {`
		1	2,3,5/3

```
        super(name, price); // Calling superclass constructor
        this.isVegetarian = isVegetarian;
    }

    // Method to display food item details
    public void displayFoodDetails() {
        super.displayDetails(); // Calling final method from superclass
        System.out.println("Vegetarian: " + (isVegetarian ? "Yes" : "No"));
    }
}

// Derived class representing a drink item
class DrinkItem extends MenuItem {
    boolean isAlcoholic;

    // Constructor
    public DrinkItem(String name, double price, boolean isAlcoholic) {
        super(name, price); // Calling superclass constructor
        this.isAlcoholic = isAlcoholic;
    }

    // Method to display drink item details
    public void displayDrinkDetails() {
        super.displayDetails(); // Calling final method from superclass
        System.out.println("Alcoholic: " + (isAlcoholic ? "Yes" : "No"));
    }
}

// Main class to test the restaurant ordering system
public class RestaurantOrderingSystem {
    public static void main(String[] args) {
        // Creating an array of MenuItem references
        MenuItem[] menu = new MenuItem[4];

        // Storing objects of FoodItem and DrinkItem in the array
        menu[0] = new FoodItem("Pasta", 12.99, true);
        menu[1] = new FoodItem("Chicken Burger", 10.99, false);
        menu[2] = new DrinkItem("Coke", 2.99, false);
        menu[3] = new DrinkItem("Wine", 15.99, true);

        // Using dynamic method dispatch to call displayDetails()
        for (MenuItem item : menu) {
            item.displayDetails();

            // Checking instance type to call specific methods
            if (item instanceof FoodItem) {
                ((FoodItem) item).displayFoodDetails();
            } else if (item instanceof DrinkItem) {
                ((DrinkItem) item).displayDrinkDetails();
            }
            System.out.println(); // Line break for clarity
        }
    }
```

	}		
5.	Design and implement a vehicle management system using **inheritance, overloading, and static methods.** The program should have a base class **Vehicle** and two derived classes: **Car and Truck.** The system should allow the user to manage and display information about different types of vehicles. **A. Create a base class Vehicle** with the following: a. **Attributes: String make, String model, and int year.** b. **Methods:** i. void displayDetails(): Displays basic vehicle information (make, model, year). ii. **Overload displayDetails** to accept a boolean parameter detailed. If detailed is true, display additional vehicle-specific information. iii. Add a **static method void showType()** to display the type of class (e.g., "Vehicle"). **B. Create a derived class Car** with the following: a. **Attribute: int numberOfDoors.** b. **Methods:** i. Override **displayDetails()** to include the number of doors. ii. **Overload displayDetails** to display details with a custom format. iii. Add a **static method void showType()** to display the type of class (e.g., "Car"). **C. Create another derived class Truck** with the following: a. **Attribute: double loadCapacity** (in tons). b. **Methods:** i. Override **displayDetails()** to include the load capacity. ii. **Overload displayDetails** to display details with a custom format. iii. Add a **static method void showType()** to display the type of class (e.g., "Truck"). **D.** In the **main** method Create an array of **Vehicle** references and store objects of both **Car and Truck.** Use **dynamic method dispatch** to call the **displayDetails()** method for each object, displaying the details specific to the type of vehicle. Call the static method **showType()** on each class to demonstrate its usage.	1	2,3,5/3

```
import java.util.ArrayList;

// Base class Vehicle
class Vehicle {
    protected String make, model;
    protected int year;

    public Vehicle(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    public void displayDetails() {
        System.out.println("Make: " + make + ", Model: " + model + ", Year: " + year);
    }

    public void displayDetails(boolean detailed) {
        displayDetails();
        if (detailed) {
            System.out.println("Additional details not available for generic vehicle.");
```

```
        }
    }

    public static void showType() {
        System.out.println("Vehicle");
    }
}

// Derived class Car
class Car extends Vehicle {
    private int numberOfDoors;

    public Car(String make, String model, int year, int numberOfDoors) {
        super(make, model, year);
        this.numberOfDoors = numberOfDoors;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Number of Doors: " + numberOfDoors);
    }

    public void displayDetails(String format) {
         System.out.println(format + "Car - Make: " + make + ", Model: " + model + ", Year:
" + year + ", Doors: " + numberOfDoors);
    }

    public static void showType() {
        System.out.println("Car");
    }
}

// Derived class Truck
class Truck extends Vehicle {
    private double loadCapacity;

    public Truck(String make, String model, int year, double loadCapacity) {
        super(make, model, year);
        this.loadCapacity = loadCapacity;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Load Capacity: " + loadCapacity + " tons");
    }

    public void displayDetails(String format) {
          System.out.println(format + "Truck - Make: " + make + ", Model: " + model + ",
Year: " + year + ", Load Capacity: " + loadCapacity + " tons");
    }
```

```
    public static void showType() {
        System.out.println("Truck");
    }
}

// Main class
public class VehicleManagementSystem {
    public static void main(String[] args) {
        Vehicle[] vehicles = new Vehicle[3];
        vehicles[0] = new Car("Toyota", "Camry", 2022, 4);
        vehicles[1] = new Truck("Ford", "F-150", 2023, 5.0);
        vehicles[2] = new Car("Honda", "Civic", 2021, 2);

        for (Vehicle vehicle : vehicles) {
            vehicle.displayDetails();
            System.out.println();
        }

        Vehicle.showType();
        Car.showType();
        Truck.showType();
    }
}
```

6. Create an online shopping Class with the following requirements demonstrating the usage of **packages and customized exception handling:** — 2 — 2,3,5/3

A. Create a package named **shopping.**
B. Define an interface **Product** with methods **getPrice(), getDetails(),** and **printDetails().**
C. Create a base class **Item** that implements Product with common properties like **productId, productName,** and **price.**
D. Create two subclasses:
    a. **Electronics:** Adds properties like **warranty** and **brand**. Override **getDetails()** to include these.
    b. **Clothing:** Adds properties like **size** and **material.** Override **getDetails()** to include these.
E. Use another class **Cart** that allows adding products and calculates the total price of items.
F. Handle exceptions:
    a. **ProductNotAvailableException** if the product added to the cart is not in stock.
    b. **InvalidProductException** if an invalid product is added.
G. Demonstrate the functionality with a **main()** method in a package **application,** ensuring all features are tested.

```
package shopping;

import java.util.ArrayList;
```

```
// Define Product interface
interface Product {
    double getPrice();
    String getDetails();
    void printDetails();
}

// Base class Item implementing Product
abstract class Item implements Product {
    protected String productId, productName;
    protected double price;

    public Item(String productId, String productName, double price) {
        this.productId = productId;
        this.productName = productName;
        this.price = price;
    }

    @Override
    public double getPrice() {
        return price;
    }

    @Override
    public void printDetails() {
        System.out.println(getDetails());
    }
}

// Subclass Electronics
class Electronics extends Item {
    private String brand;
    private int warranty;

    public Electronics(String productId, String productName, double price, String brand,
int warranty) {
        super(productId, productName, price);
        this.brand = brand;
        this.warranty = warranty;
    }

    @Override
    public String getDetails() {
        return "Electronics - " + productName + " (Brand: " + brand + ", Warranty: " +
warranty + " years, Price: $" + price + ")";
    }
}

// Subclass Clothing
class Clothing extends Item {
    private String size, material;
```

```
    public Clothing(String productId, String productName, double price, String size,
String material) {
        super(productId, productName, price);
        this.size = size;
        this.material = material;
    }

    @Override
    public String getDetails() {
        return "Clothing - " + productName + " (Size: " + size + ", Material: " + material +
", Price: $" + price + ")";
    }
}

// Exception classes
class ProductNotAvailableException extends Exception {
    public ProductNotAvailableException(String message) {
        super(message);
    }
}

class InvalidProductException extends Exception {
    public InvalidProductException(String message) {
        super(message);
    }
}

// Shopping Cart
class Cart {
    private ArrayList<Product> cartItems;

    public Cart() {
        cartItems = new ArrayList<>();
    }

    public void addProduct(Product product) throws ProductNotAvailableException,
InvalidProductException {
        if (product == null) {
            throw new InvalidProductException("Invalid product! Cannot add to cart.");
        }
        cartItems.add(product);
    }

    public double calculateTotal() {
        return cartItems.stream().mapToDouble(Product::getPrice).sum();
    }

    public void displayCart() {
        if (cartItems.isEmpty()) {
            System.out.println("Cart is empty.");
            return;
        }
        for (Product product : cartItems) {
```

```
            product.printDetails();
        }
        System.out.println("Total Price: $" + calculateTotal());
    }
}

// Main application in a different package
package application;

import shopping.*;

public class OnlineShoppingApp {
    public static void main(String[] args) {
        try {
            Cart cart = new Cart();
            Product laptop = new Electronics("E1001", "Laptop", 1200, "Dell", 2);
            Product tshirt = new Clothing("C2001", "T-Shirt", 25, "M", "Cotton");

            cart.addProduct(laptop);
            cart.addProduct(tshirt);

            System.out.println("\nShopping Cart:");
            cart.displayCart();
        } catch (ProductNotAvailableException | InvalidProductException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

7.	Design and implement a **University Management System** demonstrating **abstract classes** and **interfaces** with the following features: A. **Create a Package university:** a. **Interface Person:** Methods: **getDetails()** and **printDetails().** B. **Create a Base Class Employee:** a. Implements the **Person** interface. b. Attributes: **name** (String): Employee's name, **age** (int): Employee's age, **designation** (String): Employee's job title, **salary** (double): Employee's salary. C. **Create Two Subclasses:** a. **Professor:** i. Additional attributes: **department** (String): Department, **researchArea** (String): Research area. ii. Override **getDetails()** and **printDetails()** to include the new attributes. b. **AdminStaff:** i. Additional attributes: **role** (String): Role in the administration, **workingHours** (int): Number of working hours.	1	2,3,5/3

ii. Override **getDetails()** and **printDetails()** to include the new attributes.

D. **Main Method:** Create objects for **Professor** and **AdminStaff.** Prompt the user to enter details for each object. Display the entered details for both **Professor** and **AdminStaff.**

```
package university;

// Define Person interface
interface Person {
    String getDetails();
    void printDetails();
}

// Base class Employee implementing Person
abstract class Employee implements Person {
    protected String name, designation;
    protected int age;
    protected double salary;

    public Employee(String name, int age, String designation, double salary) {
        this.name = name;
        this.age = age;
        this.designation = designation;
        this.salary = salary;
    }

    @Override
    public void printDetails() {
        System.out.println(getDetails());
    }
}

// Subclass Professor
class Professor extends Employee {
    private String department, researchArea;

    public Professor(String name, int age, double salary, String department, String
researchArea) {
        super(name, age, "Professor", salary);
        this.department = department;
        this.researchArea = researchArea;
    }

    @Override
    public String getDetails() {
        return "Professor - Name: " + name + ", Age: " + age + ", Salary: $" + salary + ",
Department: " + department + ", Research Area: " + researchArea;
    }
}

// Subclass AdminStaff
```

```
class AdminStaff extends Employee {
    private String role;
    private int workingHours;

    public AdminStaff(String name, int age, double salary, String role, int workingHours)
{
        super(name, age, "Admin Staff", salary);
        this.role = role;
        this.workingHours = workingHours;
    }

    @Override
    public String getDetails() {
        return "Admin Staff - Name: " + name + ", Age: " + age + ", Salary: $" + salary + ",
Role: " + role + ", Working Hours: " + workingHours;
    }
}

// Main application in a different package
package application;

import university.*;

public class UniversityManagementApp {
    public static void main(String[] args) {
        Professor prof = new Professor("Dr. John Smith", 45, 90000, "Computer Science",
"Artificial Intelligence");
        AdminStaff admin = new AdminStaff("Alice Brown", 38, 60000, "HR Manager",
40);

        System.out.println("University Employees:");
        prof.printDetails();
        admin.printDetails();
    }
}
```

8.

Design a Java banking application demonstrating **System-defined Exceptions**, and **User Defined Exception handling**, with the following requirements:

2 | 2,3,5/3

A. Create a Package banking:

- **Define an Interface Account:**
  - Methods: **void deposit(double amount), void withdraw(double amount), double checkBalance()**
- **Implement the Interface in a Class BankAccount:**
  - Common Properties: **String accountNumber, String accountHolder, double balance**
  - **Constructor:** Initialize account details and balance.
  - **Methods:** Implement the methods from **Account.**
- **Create Subclasses of BankAccount:**
  - **SavingsAccount:**
    - Additional Property: **double interestRate.**

- Method: **double calculateInterest():** Calculate interest using the formula:Interest=balance×interestRate×timePeriod
- Override **withdraw()** to throw a custom **InsufficientFundsException** if the withdrawal amount exceeds the balance.

○ **CurrentAccount:**

- Additional Property: **double overdraftLimit.**
- Override **withdraw()** to throw a custom **OverdraftLimitExceededException** if the withdrawal amount exceeds the balance and overdraft limit combined.

B. Handle Exceptions:

- System-Defined Exception: Handle invalid input using **NumberFormatException.**
- Custom Exceptions: Define **InsufficientFundsException** and **OverdraftLimitExceededException** in the **banking** package.

C. Main Class: Create objects for **SavingsAccount** and **CurrentAccount.** Perform operations like deposit, withdraw, and check balance. Use the **calculateInterest()** method for **SavingsAccount** and demonstrate mathematical calculations. Handle both system-defined and custom exceptions during operations.

```
package banking;

// Define Account interface
interface Account {
   void deposit(double amount);
   void withdraw(double amount) throws InsufficientFundsException,
OverdraftLimitExceededException;
   double checkBalance();
}

// Custom Exceptions
class InsufficientFundsException extends Exception {
   public InsufficientFundsException(String message) {
      super(message);
   }
}

class OverdraftLimitExceededException extends Exception {
   public OverdraftLimitExceededException(String message) {
      super(message);
   }
}

// BankAccount class implementing Account interface
class BankAccount implements Account {
   protected String accountNumber, accountHolder;
   protected double balance;

   public BankAccount(String accountNumber, String accountHolder, double balance) {
      this.accountNumber = accountNumber;
      this.accountHolder = accountHolder;
```

```
        this.balance = balance;
    }

    @Override
    public void deposit(double amount) {
        if (amount <= 0) {
            throw new NumberFormatException("Invalid deposit amount");
        }
        balance += amount;
    }

    @Override
    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Insufficient funds in account");
        }
        balance -= amount;
    }

    @Override
    public double checkBalance() {
        return balance;
    }
}

// SavingsAccount subclass
class SavingsAccount extends BankAccount {
    private double interestRate;

    public SavingsAccount(String accountNumber, String accountHolder, double balance,
double interestRate) {
        super(accountNumber, accountHolder, balance);
        this.interestRate = interestRate;
    }

    public double calculateInterest(double timePeriod) {
        return balance * interestRate * timePeriod;
    }
}

// CurrentAccount subclass
class CurrentAccount extends BankAccount {
    private double overdraftLimit;

    public CurrentAccount(String accountNumber, String accountHolder, double balance,
double overdraftLimit) {
        super(accountNumber, accountHolder, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public void withdraw(double amount) throws InsufficientFundsException,
OverdraftLimitExceededException {
```

```
        if (amount > balance + overdraftLimit) {
            throw new OverdraftLimitExceededException("Overdraft limit exceeded");
        } else if (amount > balance) {
            System.out.println("Using overdraft limit...");
            balance -= amount;
        } else {
            balance -= amount;
        }
    }
}

// Main application
package application;

import banking.*;

public class BankingApp {
    public static void main(String[] args) {
        try {
            SavingsAccount savings = new SavingsAccount("S1001", "Alice", 5000, 0.03);
            CurrentAccount current = new CurrentAccount("C2001", "Bob", 2000, 1000);

            savings.deposit(1000);
            current.deposit(500);

            System.out.println("Savings Balance: " + savings.checkBalance());
            System.out.println("Current Balance: " + current.checkBalance());

            savings.withdraw(2000);
            current.withdraw(2500);

            System.out.println("Savings Balance after withdrawal: " +
savings.checkBalance());
            System.out.println("Current Balance after withdrawal: " +
current.checkBalance());

            double interest = savings.calculateInterest(2);
            System.out.println("Interest earned on Savings Account: " + interest);
        } catch (NumberFormatException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (InsufficientFundsException | OverdraftLimitExceededException e) {
            System.out.println("Transaction Error: " + e.getMessage());
        }
    }
}
```

	PART - B		

1.	Create a simple library management system where multiple users (threads) can borrow and return books at the same time by implementing **Runnable Interface**.
	
	A. **Create a Book class**:
	a. The class should have the following attributes: **bookTitle** (String) - title of the book, **author** (String) - author of the book, **availability** (boolean) - a flag to indicate whether the book is available for borrowing.
	b. Implement the following methods in the Book class:
	i. **borrow()** - Allows a user to borrow the book. If the book is already borrowed, it should return **false.**
	ii. **returnBook()** - Marks the book as available again after returning it.
	B. **Create a User class that implements the Runnable interface**:
	a. Each **User** represents a thread trying to borrow and return a book.
	b. Each **User** should: Attempt to borrow a book using the **borrow()** method, Simulate reading the book by sleeping for a random amount of time, Return the book using the **returnBook()** method.
	C. **In the main method**: Create a **Book** object. Create multiple **User** threads and assign them the task of borrowing and returning the book. Start the threads and use the **join()** method to ensure that all threads finish their execution before the program terminates.
	
	`class Book {`
	`  private final String title;`
	`  private boolean available = true;`
	
	`  public Book(String title) { this.title = title; }`
	
	`  public synchronized boolean borrow(String user) {`
	`  if (!available) return false;`
	`  available = false;`
	`  System.out.println(user + " borrowed " + title);`
	`  return true;`
	`  }`
	
	`  public synchronized void returnBook(String user) {`
	`  available = true;`
	`  System.out.println(user + " returned " + title);`
	`  }`
	`}`
	
	`class User implements Runnable {`
	`  private final String name;`
	`  private final Book book;`
	
	`  public User(String name, Book book) {`
	`  this.name = name;`
		2	2,3,5/3

	this.book = book;
	}
	
	public void run() {
	if (book.borrow(name)) {
	try { Thread.sleep((int) (Math.random() * 3000) + 1000); } catch (InterruptedException ignored) {}
	book.returnBook(name);
	} else {
	System.out.println(name + " couldn't borrow " + book);
	}
	}
	}
	
	public class LibrarySystem {
	public static void main(String[] args) throws InterruptedException {
	Book book = new Book("Effective Java");
	Thread[] users = { new Thread(new User("Alice", book)), new Thread(new User("Bob", book)), new Thread(new User("Charlie", book)) };
	for (Thread user : users) user.start();
	for (Thread user : users) user.join();
	System.out.println("All users have completed borrowing and returning.");
	}
	}		

2.	Create a Java program that contains a **JFrame** with a **JButton** and a **JLabel**. When the button is clicked, it should change the text of the label and also change the background color of the frame.
	
	● Use **ActionListener** to handle the button click event.
	● The button should change the text of the label to **"Button Clicked!".**
	● The background color of the frame should change to a random color each time the button is clicked.
	
	import javax.swing.*;
	import java.awt.*;
	import java.awt.event.*;
	import java.util.Random;
	
	public class SimpleGUI extends JFrame {
	private JLabel label;
	private JButton button;
	private Random random = new Random();
	
	public SimpleGUI() {
	setTitle("Simple GUI");
	setSize(300, 200);
	setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
	setLayout(new FlowLayout());
	
	label = new JLabel("Click the button!");
	button = new JButton("Click Me");
		3	2,3,5/3

```
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                label.setText("Button Clicked!");
                getContentPane().setBackground(new Color(random.nextInt(256),
random.nextInt(256), random.nextInt(256)));
            }
        });

        add(label);
        add(button);

        setVisible(true);
    }

    public static void main(String[] args) {
        new SimpleGUI();
    }
}
```

3.	Implement a **task scheduling system** where multiple tasks (threads) are executed based on their priority. A. Create an abstract class **Task** with methods **execute()** and **getPriority().** B. Create two subclasses: **HighPriorityTask and LowPriorityTask,** each overriding the **execute()** method to simulate long-running tasks (e.g., printing large numbers, waiting, etc.). C. Implement a scheduling system where high-priority tasks are executed before low-priority tasks using thread priorities. D. Demonstrate the use of **isAlive()** to check if a task has completed and **join()** to wait for all tasks to finish before terminating the program.	2	2,3,5/3

```
// Abstract Task Class
abstract class Task extends Thread {
    public Task(String name, int priority) {
        super(name);
        setPriority(priority);
    }
    public abstract void execute();
    @Override public void run() { execute(); }
}

// High-Priority Task
class HighPriorityTask extends Task {
    public HighPriorityTask(String name) { super(name, Thread.MAX_PRIORITY); }
    public void execute() {
        System.out.println(getName() + " (High Priority) started.");
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " executing step " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) {}
        }
        System.out.println(getName() + " (High Priority) completed.");
    }
}
```

```
// Low-Priority Task
class LowPriorityTask extends Task {
    public LowPriorityTask(String name) { super(name, Thread.MIN_PRIORITY); }
    public void execute() {
        System.out.println(getName() + " (Low Priority) started.");
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " executing step " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) {}
        }
        System.out.println(getName() + " (Low Priority) completed.");
    }
}

// Task Scheduler Main Class
public class TaskScheduler {
    public static void main(String[] args) throws InterruptedException {
        Task[] tasks = {
            new HighPriorityTask("HighPriorityTask-1"),
            new HighPriorityTask("HighPriorityTask-2"),
            new LowPriorityTask("LowPriorityTask-1"),
            new LowPriorityTask("LowPriorityTask-2")
        };

        for (Task task : tasks) task.start();
        for (Task task : tasks) task.join();

        System.out.println("All tasks completed.");
    }
}
```

4. 3 2,3,5/3

Create a Java program that uses a **block lambda** to perform an operation involving multiple steps. Specifically:

1. Write a functional interface **StringProcessor** with a method **process(String input).**
2. Implement a block lambda that: Reverses the input string, Convert it to uppercase, Returns the final modified string.
3. Demonstrate the block lambda by applying it to an array of strings and printing the results.

```
// Functional interface
interface StringProcessor {
    String process(String input);
}

public class BlockLambdaExample {
    public static void main(String[] args) {
        // Block lambda implementation
        StringProcessor processor = (input) -> {
            String reversed = new StringBuilder(input).reverse().toString();
            return reversed.toUpperCase();
        };
```

```
        // Array of strings
        String[] strings = { "hello", "java", "lambda", "world" };

        // Process each string and print the result
        for (String str : strings) {
            System.out.println("Original: " + str + " | Processed: " + processor.process(str));
        }
    }
}
```

5. Design a program to demonstrate **synchronization** of an ATM **system** where multiple customers (threads) can perform transactions (deposit and withdraw) on a shared ATM. (2) (2,3,5/3)

   A. **ATM Class:**
      a. Methods:
         i. **withdraw(double amount):** Withdraw money.
         ii. **deposit(double amount):** Deposit money.
         iii. **getBalance():** Get current balance.
   B. **Customer Class:** Each customer (thread) will perform a series of transactions: deposit, withdraw, and check balance.
   C. Use synchronization to ensure that only one thread accesses the ATM's balance at a time.
   D. Implement thread suspension, resumption, and stopping. Use **isAlive()** and **join()** to manage thread execution and ensure all customers complete their transactions.

```
// ATM class with synchronized deposit and withdraw methods
class ATM {
    private double balance;

    public ATM(double initialBalance) {
        this.balance = initialBalance;
    }

    // Synchronized withdraw method
    public synchronized void withdraw(double amount, String customerName) {
        if (amount > balance) {
            System.out.println(customerName + " attempted to withdraw $" + amount + " but
insufficient balance!");
        } else {
            balance -= amount;
              System.out.println(customerName + " withdrew $" + amount + ". New balance:
$" + balance);
        }
    }

    // Synchronized deposit method
    public synchronized void deposit(double amount, String customerName) {
        balance += amount;
          System.out.println(customerName + " deposited $" + amount + ". New balance: $"
+ balance);
    }
```

```
    // Get balance
    public synchronized double getBalance() {
        return balance;
    }
}

// Customer class that extends Thread
class Customer extends Thread {
    private ATM atm;
    private String name;

    public Customer(ATM atm, String name) {
        this.atm = atm;
        this.name = name;
    }

    @Override
    public void run() {
        atm.deposit(500, name);
        atm.withdraw(200, name);
        System.out.println(name + " checked balance: $" + atm.getBalance());
    }
}

// Main class to run the program
public class ATMSynchronization {
    public static void main(String[] args) {
        ATM atm = new ATM(1000); // Initialize ATM with $1000

        // Create customer threads
        Customer c1 = new Customer(atm, "Alice");
        Customer c2 = new Customer(atm, "Bob");

        // Start threads
        c1.start();
        c2.start();

        // Use join() to ensure main thread waits for customers to finish
        try {
            c1.join();
            c2.join();
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted!");
        }

            System.out.println("All transactions completed. Final ATM balance: $" +
atm.getBalance());
    }
}
```

6.	Create a Java program that fulfills the following requirements:
	1. The user can draw lines on a **JPanel** by clicking and dragging the mouse.
	2. As the user drags the mouse while holding the button, the program should continuously draw a line.
	3. Use a **MouseAdapter** class to handle mouse events.
	`import javax.swing.*;`
	`import java.awt.*;`
	`import java.awt.event.*;`
	
	`public class SimpleDrawing extends JPanel {`
	`  private int startX, startY;`
	
	`  public SimpleDrawing() {`
	`  addMouseListener(new MouseAdapter() {`
	`  public void mousePressed(MouseEvent e) {`
	`  startX = e.getX();`
	`  startY = e.getY();`
	`  }`
	`  });`
	
	`  addMouseMotionListener(new MouseMotionAdapter() {`
	`  public void mouseDragged(MouseEvent e) {`
	`  Graphics g = getGraphics();`
	`  g.drawLine(startX, startY, e.getX(), e.getY());`
	`  startX = e.getX();`
	`  startY = e.getY();`
	`  }`
	`  });`
	`  }`
	
	`  public static void main(String[] args) {`
	`  JFrame frame = new JFrame("Draw Lines");`
	`  frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`
	`  frame.setSize(600, 400);`
	`  frame.add(new SimpleDrawing());`
	`  frame.setVisible(true);`
	`  }`
	`}`
		3	2,3,5/3

7.	Write a Java program to filter and print names from a list that start with the letter 'A' using Lambda expressions.
	
	`import java.util.Arrays;`
	`import java.util.List;`
	`import java.util.stream.Collectors;`
	
	`public class FilterNames {`
	`  public static void main(String[] args) {`
	`  List<String> names = Arrays.asList("Alice", "Bob", "Anna", "Charlie", "Andrew", "David", "Amanda");`
	
	`  // Using lambda expression to filter names that start with 'A'`
	`  List<String> filteredNames = names.stream()`
	`  .filter(name -> name.startsWith("A"))`
		3	2,3,5/3

```
                .collect(Collectors.toList());

        // Print the filtered names
        System.out.println("Names starting with 'A': " + filteredNames);
    }
}
```

8. Design a Java Swing-based calculator application that performs basic arithmetic operations (Addition, Subtraction, Multiplication, Division). Include a clear button to reset inputs. — 3 — 2,3,5/3

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class CalculatorApp extends JFrame implements ActionListener {
    private JTextField textField;
    private double num1, num2, result;
    private char operator;

    public CalculatorApp() {
        setTitle("Swing Calculator");
        setSize(400, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Text field for input/output
        textField = new JTextField();
        textField.setFont(new Font("Arial", Font.BOLD, 24));
        textField.setHorizontalAlignment(JTextField.RIGHT);
        textField.setEditable(false);
        add(textField, BorderLayout.NORTH);

        // Button panel
        JPanel panel = new JPanel(new GridLayout(4, 4, 5, 5));
        String[] buttons = {
            "7", "8", "9", "/",
            "4", "5", "6", "*",
            "1", "2", "3", "-",
            "C", "0", "=", "+"
        };

        for (String text : buttons) {
            JButton button = new JButton(text);
            button.setFont(new Font("Arial", Font.BOLD, 20));
            button.addActionListener(this);
            panel.add(button);
        }

        add(panel, BorderLayout.CENTER);
        setVisible(true);
    }
```

```
    @Override
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();

        if (command.matches("[0-9]")) { // Append number
            textField.setText(textField.getText() + command);
        } else if (command.matches("[+\\-*/]")) { // Store operator and first number
            num1 = Double.parseDouble(textField.getText());
            operator = command.charAt(0);
            textField.setText("");
        } else if (command.equals("=")) { // Perform calculation
            try {
                num2 = Double.parseDouble(textField.getText());
                switch (operator) {
                    case '+': result = num1 + num2; break;
                    case '-': result = num1 - num2; break;
                    case '*': result = num1 * num2; break;
                    case '/':
                        if (num2 == 0) throw new ArithmeticException("Cannot divide by zero");
                        result = num1 / num2;
                        break;
                }
                textField.setText(String.valueOf(result));
            } catch (Exception ex) {
                textField.setText("Error");
            }
        } else if (command.equals("C")) { // Clear input
            textField.setText("");
            num1 = num2 = result = 0;
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(CalculatorApp::new);
    }
}
```

Marks Distribution

Conduction and Result	Part	Write-up	Execution	Viva	Total	Change of Program
	A	5M	20M	7M	50	-8M
	B	3M	15M			